DAY 5 예지보전 · 따라 하기 매뉴얼

설비 센서 신호로 고장을 미리 읽고 공개하기

course-forge로 만든 자율학습 강의 /learn-auto-pdm

단위 점검 → 평상 범위 → 노이즈 vs 추세 → 파일럿 선정 → 대시보드 공개 → 공유

명령어를 외울 필요는 없다. 우리가 하는 일은 두 가지 —
① 노란 상자의 글을 복사해 붙여 넣기 ② 결과가 맞게 나왔는지 확인하기.

수업이 끝나면 내 예지보전 대시보드가 인터넷 주소로 열린다

https://내아이디.github.io/kma-day5-pdm/ — 휴대폰으로도 열린다

완성하면

- 설비B 온도·진동 추세 차트
- 노이즈 vs 추세 판정표
- 파일럿 순위 (프레스 2.59배)
- 국내외 도입 사례·출처
- 4단 보고서 · 공유 메시지



이 강의는 Day5 본 강의의 6단계를 그대로 따라간다

능률협회 자동차산업과 생성형 AI 3-1 · Day5 예지보전을 course-forge로 옮겼다

원본 6단계	이 강의의 블록	끝나면 생기는 것
① 환경	Block 0 준비	작업 폴더 · Day5 데이터 4종
② 데이터 (단위·주기 확인)	Block 1 단위 점검	단위 6칸 변환 기록
③ 전처리 (평균 ± 2σ)	Block 2 평상 범위	이상 후보표
④ 분석 1 (노이즈 vs 추세)	Block 3 노이즈 vs 추세	판정표 · 추세 차트
보강 실습 B (파일럿 선정)	Block 4 파일럿 선정	공정 순위 · 순효과
⑤ 분석 2 (보고서·차트)	Block 5 대시보드 공개	사이트 주소
⑥ 평가	Block 6 보고서와 공유	4단 보고서 · 공유 메시지

수업 방식

만들기 → 배우기 → 배포 → 공유가 한 바퀴다

이 강의는 course-forge로 만든 것이다. 나는 배우고, 결과를 배포하고, 동료에게 공유한다

① 만들기

/course-forge

Day5 덱·데이터로
강의 스킬을
만든다

② 배우기

/learn-auto-pdm

블록마다
설명 → 실습
→ 퀴즈

③ 배포

GitHub Pages

결과 대시보드를
내 주소로
공개한다

④ 공유

설치 한 줄

동료가 같은
수업을 듣고
의견을 준다

목표 AI Native · 방법 Skills as Curriculum · 형식 자율학습 — 스킬이 강사다

모든 단계는 어디 → 입력 → 출력 → 피드백 네 칸으로 읽는다

이 매뉴얼의 단계 슬라이드는 모두 같은 모양이다. 번호 순서대로 보면 된다

① 어디	② 입력	③ 출력	④ 피드백
어느 창에 입력하나	**무엇을 넣나 (복사 → 붙여 넣기 → Enter)**	**무엇이 나오나 (화면에 보이는 모습)**	**맞게 됐나 확인 아니면 무엇을 하나**
Code 탭 입력칸 · 브라우저	노란 상자 = 프롬프트 파란 머리 상자 = 누를 것	칸 안의 모습 = 내 화면에 나올 모습	성공 ▸ 이렇게 보이면 OK 막히면 ▸ 이렇게 한다

④ 피드백이 '성공'이 아니면 다음 단계로 넘어가지 않는다 — 막히면 칸의 지시대로 한다

꼭 알아야 할 용어는 여섯 개다

나머지 용어는 블록마다 설명 슬라이드에서 정의한다

용어	현장 정의
예지보전 (PdM)	센서로 고장 징후를 미리 감지해 정비 시점을 정한다 — 사후·예방 다음, 처방 앞 단계
평상 범위	평균 − 2σ ~ 평균 + 2σ. 벗어난 값이 이상 후보 (원본 ③ 전처리 기준)
노이즈 / 추세	한 번 튀고 돌아오는 값 / 임계선을 향해 꾸준히 오르는 흐름 — 추세가 정비 신호
기준선	정상 운전 구간(처음 12점)의 평균과 σ. 추세가 시작된 시점을 잡는 기준
파일럿	예지보전을 먼저 적용할 공정 하나 — 연간 정지손실 ÷ 초기 투자로 고른다
GitHub Pages	저장소의 index.html을 인터넷 주소로 공개하는 무료 기능

시작 전에 준비물 네 가지와 약속 세 가지를 확인한다

센서·공정 데이터는 원본 강의의 가상 데이터, 도입 사례는 공개 기사다

준비물

- 컴퓨터 (맥 macOS 13+ 또는 윈도우 10/11)
- Claude 유료 계정 (Pro·Max·Team·Enterprise)
- GitHub 계정 (Day3에서 만든 것 그대로)
- 설치 한 줄 또는 learn-auto-pdm.zip

약속 세 가지

- 비밀번호·일회용 코드는 github.com(브라우저)에만 입력한다
- 공개 저장소다 — 우리 회사 실제 설비 데이터는 넣지 않는다
- 모르면 멈추고 묻는다 — 그냥 '허용'을 누르지 않는다

01

STEP 1–2 · 설치

데스크톱 앱과 강의 스킬 설치

Day3 수업에서 앱을 설치했다면 STEP 2부터 (약 10분)

Claude 데스크톱 앱에서 Code 탭과 폴더를 고른다

앱 안에 Claude Code가 들어 있다. 윈도우는 Git for Windows를 먼저 설치한다

① 어디



Claude 데스크톱 앱 설치: claude.com/download · git-scm.com/downloads/win (윈도우)

② 입력 · 앱에서 순서대로

1. (윈도우) Git for Windows 설치 — Next 계속
2. claude.com/download 에서 앱 설치
3. Claude 계정으로 로그인
4. 위쪽 가운데 Code 탭
5. Local 선택 → 폴더 선택 → 홈 폴더
6. 아래 입력칸이 보이면 준비 끝

③ 출력 · 화면에 이렇게 나온다

```
Local · 내이름

  > 무엇을 할까요?
                              [Manual ▾] [↑]
```

④ 피드백 · 확인하고 다음으로

성공 ▸ 아래에 입력칸 · 오른쪽에 모드 선택기

막히면 ▸ '업그레이드' 안내 → 유료 계정이 필요하다

STEP 2-1

공유받은 설치 한 줄을 Claude Code에게 시킨다

zip을 받았다면 '다운로드 폴더의 learn-auto-pdm.zip을 풀어 스킬 폴더로 복사해 줘'

① 어디  **Code 탭 입력칸** Claude 데스크톱 앱 Code 탭 아래쪽 칸 — 붙여 넣고 Enter

② 입력 · 프롬프트 · Code 탭 입력칸

```
npx skills add <강의 저장소>/learn-auto-pdm
  --agent claude-code --global --yes
를 실행해 줘.

끝나면 내 홈 폴더 .claude/skills 안에
learn-auto-pdm 폴더가 있는지 확인해 줘.
```

복사: 상자 글을 마우스로 드래그 → 복사
붙여 넣기: 맥 ⌘+V · 윈도우 Ctrl+V → Enter

③ 출력 · 화면에 이렇게 나온다

```
명령을 실행할까요?  [허용] [거부]
...
✔ learn-auto-pdm 설치 확인
```

④ 피드백 · 확인하고 다음으로

성공 ▸ '설치 확인' 메시지 · 허용 카드엔 허용

막히면 ▸ npx가 없다고 하면 'Node.js를 설치해 줘'

새 세션에서 /learn 목록이 보이면 설치가 끝난 것이다

Claude Code는 세션을 시작할 때 스킬을 읽는다. 그래서 새 세션을 연다

① 어디  **Claude 데스크톱 앱** 왼쪽 사이드바의 새 세션(+)

② 입력 · 새 세션 → /learn 입력

1. 왼쪽 사이드바 새 세션(+)
2. Local → 홈 폴더
3. 입력칸에 /learn 까지만 친다
4. 목록이 뜰 때까지 기다린다

③ 출력 · 화면에 이렇게 나온다

```
> /learn
  /learn-auto-pdm
    자동차 설비 예지보전 자율학습 강의 …
```

④ 피드백 · 확인하고 다음으로

성공 ▸ 목록에 /learn-auto-pdm

막히면 ▸ 안 보이면 STEP 2-1을 다시 붙여 넣는다

02

수업 체험

Skills as Curriculum 맛보기

Block 3(노이즈 vs 추세)이 실제로 진행되는 모습을 여섯 장면으로 따라간다

블록 하나는 여섯 장면으로 진행되고 블록마다 반복된다

다음 여섯 장은 Block 3 '노이즈 vs 추세'가 실제 앱 화면에서 진행되는 모습이다

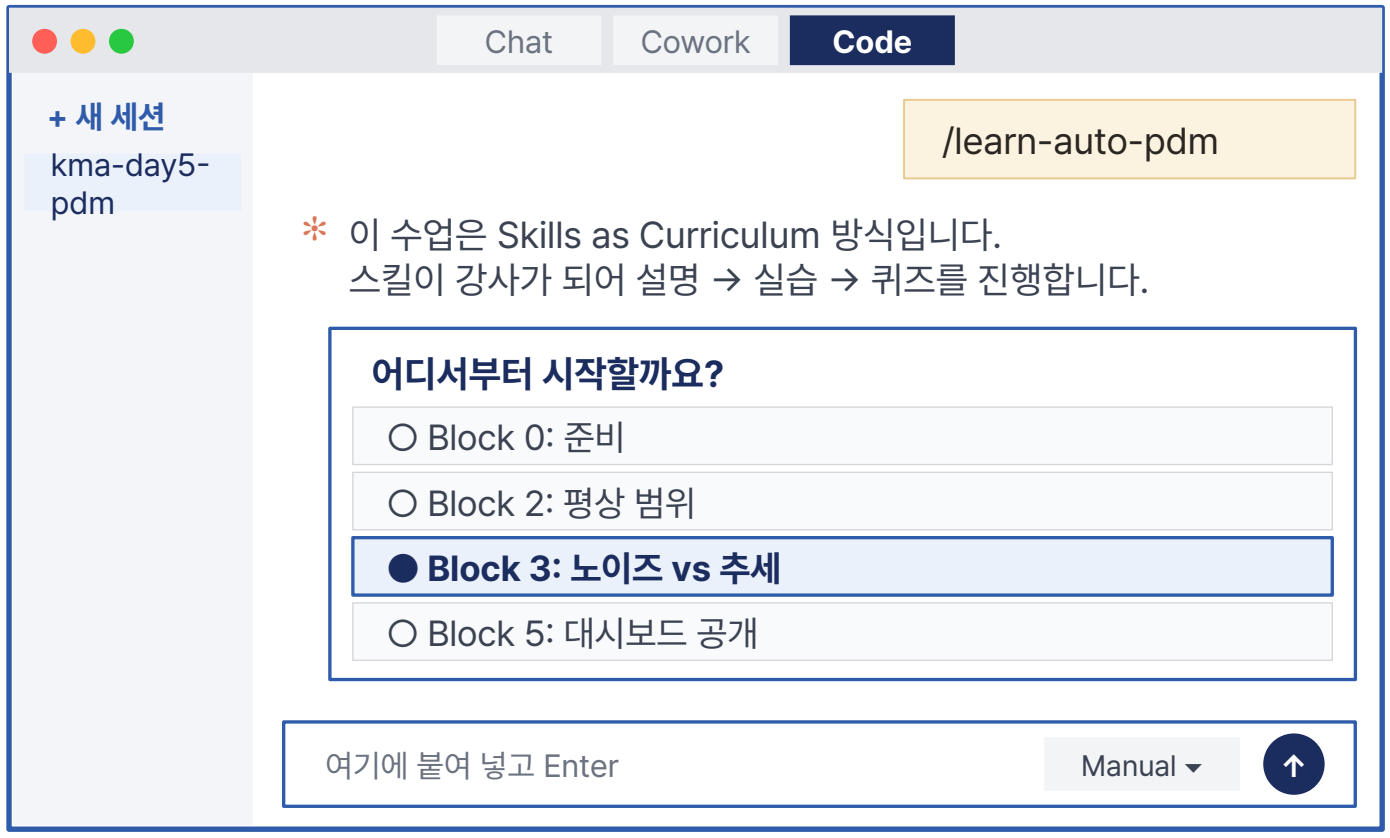
1 시작	2 설명	3 실행	4 완료	5 확인·퀴즈	6 다음
내가 한다	Claude	내가 → Claude	내가 한다	Claude	둘이 함께
스킬 이름 입력 블록 고르기	개념 설명 노란 상자 제시 그리고 멈춤	붙여 넣기 계산·차트 허용 누르기	차트를 보고 '완료' 입력	내 파일 점검 판단형 퀴즈	해설 듣기 다음 블록 → 다시 2부터

핵심 — Claude는 2 설명 뒤에 멈춘다. 내가 붙여 넣고 '완료'를 쳐야 수업이 진행된다

스킬을 부르면 Claude가 시작 블록을 묻는다

작업 폴더(kma-day5-pdm)를 연 세션에서 시작한다

1 시작 | 2 설명 | 3 실행 | 4 완료 | 5 확인·퀴즈 | 6 다음



지금 일어나는 일

- 스킬 이름을 넣으면 수업이 열린다
- Claude가 수업 방식을 소개한다
- 블록 카드 중 하나를 고른다 (처음이면 Block 0)

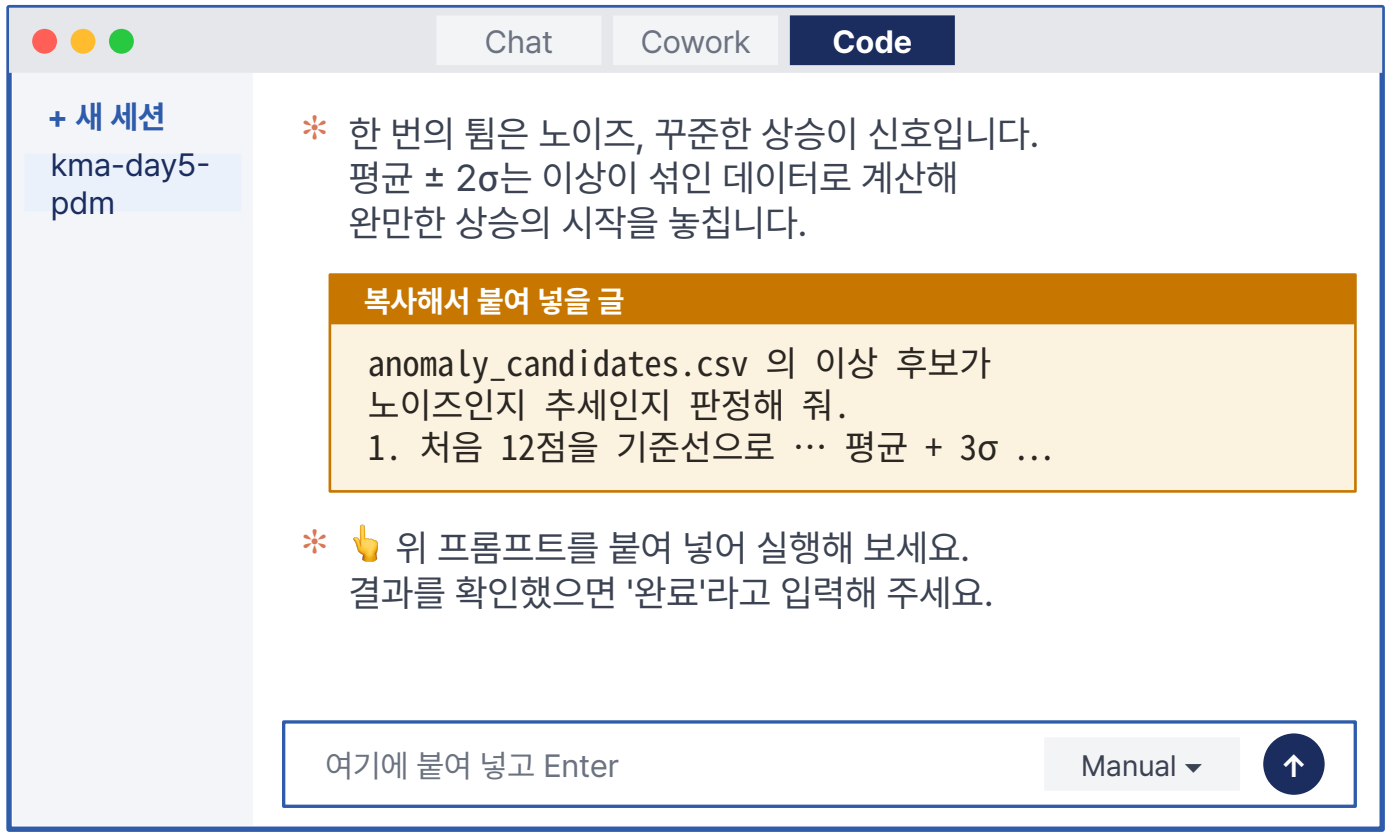
내가 할 일

/learn-auto-pdm 입력 → 블록을 눌러 고른다

Claude가 설명하고 붙여 넣을 글을 준 뒤 멈춘다

이 상태에서 기다리면 아무 일도 일어나지 않는다. 내 차례다

1 시작 | **2 설명** | 3 실행 | 4 완료 | 5 확인·퀴즈 | 6 다음



지금 일어나는 일

- 원본 덱의 메시지로 개념을 설명한다
- 노란 상자 = 내가 복사할 글
- '완료라고 입력' = 멈춤 신호

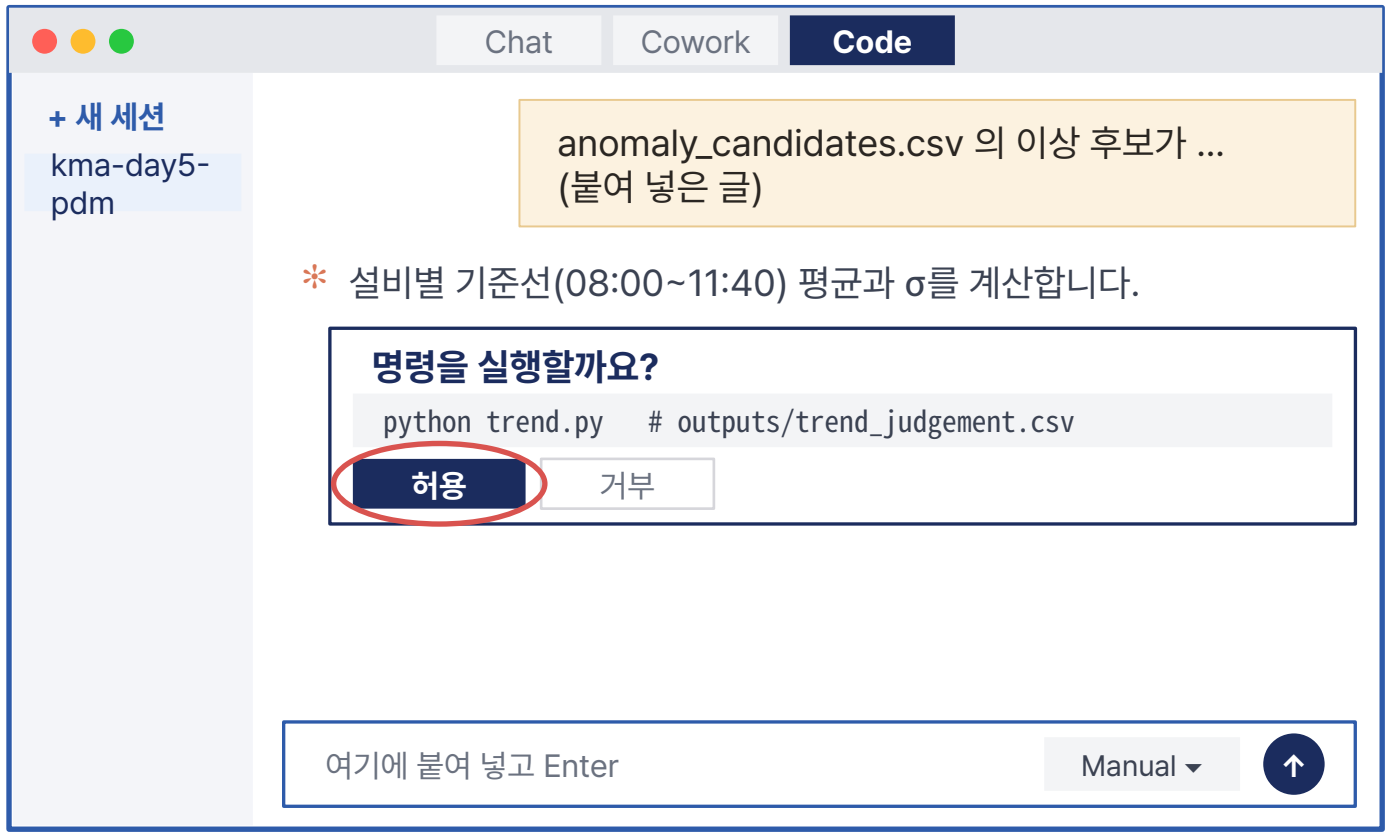
내가 할 일

노란 상자 글을 드래그해서 복사한다

붙여 넣으면 Claude가 계산하고 차트를 그린다

허용 카드에는 실행할 명령이 한 줄로 적혀 있다. 읽고 누른다

1 시작 | 2 설명 | **3 실행** | 4 완료 | 5 확인·퀴즈 | 6 다음



지금 일어나는 일

- 붙여 넣은 글이 오른쪽 말풍선
- Claude가 코드를 만들어 실행한다
- 실행 전에 허용 카드 (여러 번 나올 수 있다)

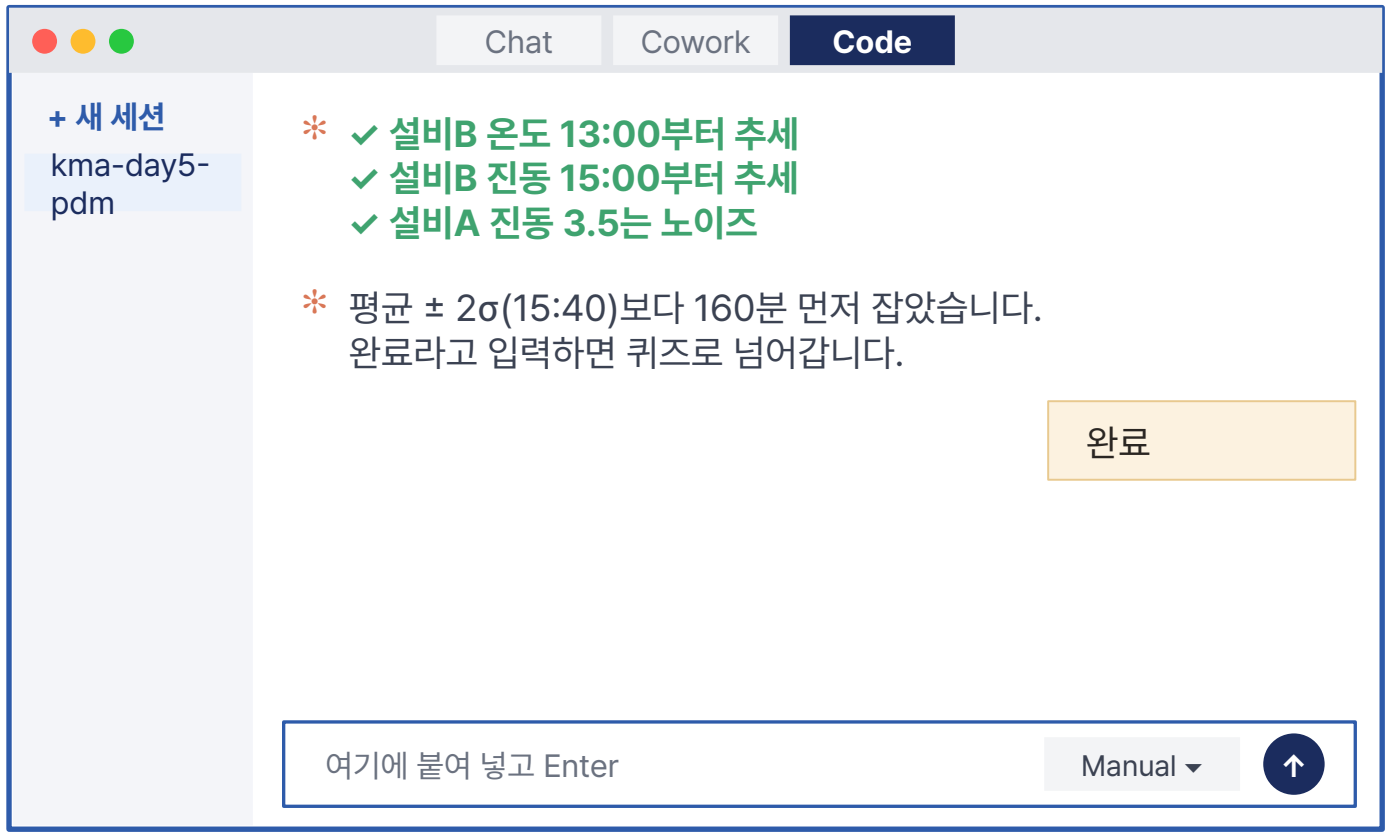
내가 할 일

입력칸에 Ctrl+V(맥 ⌘+V) → Enter → [허용]

판정 결과와 차트를 확인하고 '완료'를 입력한다

Claude는 내가 '완료'를 칠 때까지 다음으로 넘어가지 않는다

1 시작	2 설명	3 실행	**4 완료**	5 확인·퀴즈	6 다음



지금 일어나는 일

- 판정을 체크 목록으로 보고
- 온도가 진동보다 2시간 먼저 반응
- 차트가 대화창에 뜬다

내가 할 일

차트를 보고 → '완료' 입력

Claude가 내 결과를 점검한 뒤 판단형 퀴즈를 낸다

원본 강의 실습과제 2와 같은 판단을 묻는다. 점수는 없다

1 시작	2 설명	3 실행	4 완료	**5 확인·퀴즈**	6 다음



지금 일어나는 일

- Claude가 내 파일을 직접 점검
- 빠진 것이 있으면 고칠 글을 준다
- 퀴즈 카드 — 점수는 없다

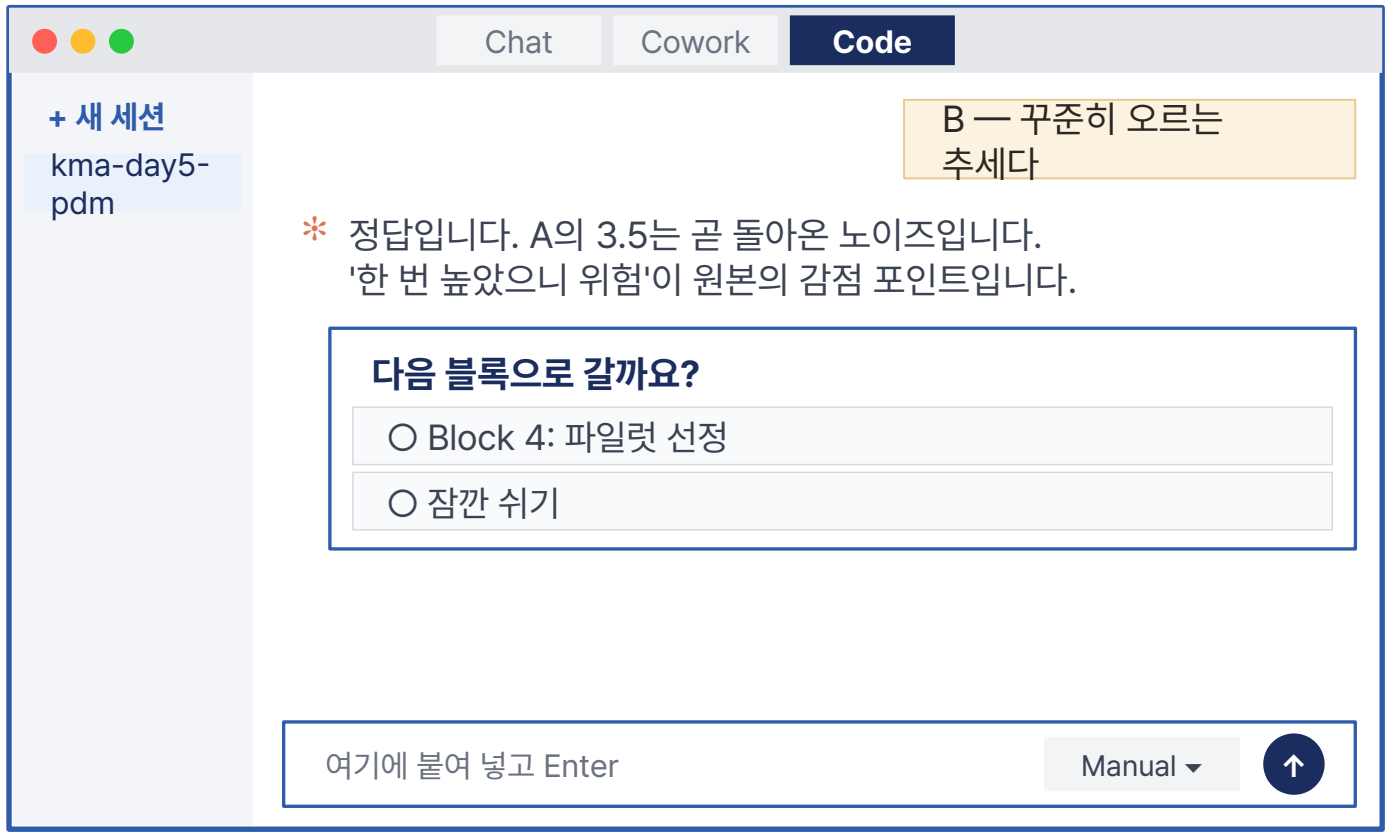
내가 할 일

보기 하나를 눌러 고른다

해설을 듣고 다음 블록으로 넘어간다

다음 블록을 고르면 장면 2(설명)부터 다시 시작한다

1 시작	2 설명	3 실행	4 완료	5 확인·퀴즈	**6 다음**



지금 일어나는 일

- 오답이어도 해설로 배운다
- 쉬었다 와도 세션이 남아 있다
- 다음 블록 → 다시 장면 2부터

내가 할 일

다음 블록을 눌러 고른다 → 장면 2부터 반복

03

BLOCK 0–2 · 데이터

준비 · 단위 점검 · 평상 범위

원본 ① 환경 → ② 데이터 → ③ 전처리

작업 폴더와 Day5 데이터 준비는 프롬프트 하나로 끝난다

Claude가 실제 스킬 경로를 채운 상자를 보여 준다. 그 상자를 복사한다

① 어디  **Code 탭 입력칸** Claude 데스크톱 앱 Code 탭 아래쪽 칸 — 붙여 넣고 Enter

② 입력 · 프롬프트 · Code 탭 입력칸

```
예지보전 실습 준비를 해 줘.
1. 홈 폴더에 kma-day5-pdm/data 를 만들고
   (스킬)/assets/lab 의 csv 4개를 복사해 줘.
2. Python·pandas 확인, 없으면 설치.
3. git 확인 후 git init.
4. CLAUDE.md에 규칙 네 줄:
   원본 보존 · 단위 먼저 확인 ·
   가상/추정 표기 · 회사 데이터 금지
5. 파일별 행 수 → env_check.txt
```

③ 출력 · 화면에 이렇게 나온다

```
설비센서_50건        50행  OK
단위혼재_함정        50행  OK
파일럿평가_가정값     4행  OK
도입사례_실데이터     6행  OK
```

④ 피드백 · 확인하고 다음으로

성공 ▸ env_check.txt 항목이 모두 OK

막히면 ▸ 권한 창이 뜨면 [예] · 맥은 비밀번호

작업 폴더로 새 세션을 열어 수업을 이어 간다

이 폴더가 그대로 GitHub 저장소가 된다. 세션마다 폴더가 정해진다

① 어디  **Claude 데스크톱 앱** 왼쪽 사이드바의 새 세션(+)

② 입력 · 새 세션 → 폴더 → 수업 다시 시작

1. 왼쪽 사이드바 새 세션(+)
2. Local → 폴더 선택
3. 홈 폴더 안의 kma-day5-pdm 선택
4. 입력칸에 /learn-auto-pdm
5. 시작 블록에서 Block 1 선택

③ 출력 · 화면에 이렇게 나온다

```
kma-day5-pdm

어디서부터 시작할까요?
  ● Block 1: 단위 점검
```

④ 피드백 · 확인하고 다음으로

성공 ▸ 사이드바 세션 이름이 kma-day5-pdm

막히면 ▸ 폴더가 안 보이면 BLOCK 0 ①을 다시 실행

튀는 값이 고장인지 단위인지는 '기록장치' 열이 알려 준다

원본 ② 데이터 단계 — 단위(°C·mm/s·bar)와 측정 주기를 먼저 확인한다

보이는 모습	고장으로 읽으면	단위로 읽으면
설비A 온도 51 → 123.6 → 124.7 → 51	20분 만에 70°C 상승	로거2(교체품) °F — 50.9°C
설비B 진동 1.05 → 0.043 → 1.12	진동이 갑자기 사라짐	로거4(휴대용) in/s — 1.09 mm/s
설비B 압력 118 → 1690 → 118	압력 14배 급등	로거4 psi — 116.5 bar

지우지 말고 바꾼다 — 지우면 그 시각이 비어 추세 판단에 구멍이 생긴다. 변환 기록을 남긴다

단위가 다른 칸을 찾아 바꾸고 변환 기록을 남긴다

바꾼 값이 정상 기록과 1% 안으로 맞으면 같은 측정값이었다는 뜻이다

① 어디



Code 탭 입력칸 Claude 데스크톱 앱 Code 탭 아래쪽 칸 — 붙여 넣고 Enter

② 입력 · 프롬프트 · Code 탭 입력칸

```
단위혼재 파일의 단위를 점검해 줘. 원본은 두고.
1. 설비별로 물리적으로 맞지 않게 튀는
   칸을 찾아 줘.
2. 그 칸의 '기록장치'를 같이 보여 주고
   °F·in/s·psi 중 무엇인지 판단해 줘.
3. ℃·mm/s·bar로 바꿔 sensor_clean.csv,
   바꾼 칸은 unit_fix_log.csv 에.
4. 50건 파일의 같은 시각 값과 차이(%).
```

③ 출력 · 화면에 이렇게 나온다

```
바꾼 칸 6개
  °F 3 (설비A 10:00~10:40, 로거2)
  in/s 2 · psi 1 (설비B, 로거4)
123.6°F → 50.89℃ · 최대 차이 0.9%
```

④ 피드백 · 확인하고 다음으로

성공 ▸ 6칸 · 지운 행 없음 (50행 그대로)

막히면 ▸ 행을 지웠다면 '지우지 말고 단위를

막히면 ▸ 바꾸는 방식으로 다시 해 줘'

평상 범위는 설비 × 센서마다 따로 잡는다

원본 ③ 전처리 — 평균 ± 2표준편차를 벗어난 값을 이상 후보로 표시한다

평상 범위

- 평균 − 2σ ~ 평균 + 2σ
- 벗어난 값 = 이상 후보 (살펴볼 값)
- 이상 후보 ≠ 정비 신호 — Block 3에서 가린다

설비를 합치면

- 설비A 약 51°C · 설비B 약 55°C
- 합쳐 계산하면 두 설비 차이가 σ에 들어간다
- 범위가 넓어져 한 설비의 이상이 묻힌다

결과 — 설비A 1점(12:40 진동 3.5) · 설비B 3점(진동 2.2·2.9, 온도 65.1)

설비별 평상 범위를 계산해 이상 후보를 표시한다

설비를 합쳐 계산했을 때와 비교하는 한 줄도 받는다

① 어디  **Code 탭 입력칸** Claude 데스크톱 앱 Code 탭 아래쪽 칸 — 붙여 넣고 Enter

② 입력 · 프롬프트 · Code 탭 입력칸

```
50건 파일로 평상 범위를 계산해 줘.
1. 설비별 [시간 / 진동 / 온도 / 압력] 표.
2. 설비 × 센서마다 평균·표준편차와
   평상 범위(평균 ± 2표준편차).
3. 벗어난 값을 '이상 후보'로 표시해
   anomaly_candidates.csv 로 저장.
4. 설비를 합쳐 계산하면 어떻게 달라지는지
   한 줄로 비교해 줘.
```

③ 출력 · 화면에 이렇게 나온다

```
설비A  12:40 진동 3.5  (범위 0.09~2.10)
설비B  15:40 진동 2.2
       16:00 진동 2.9 · 온도 65.1
```

④ 피드백 · 확인하고 다음으로

성공 ▸ 설비A 1점 · 설비B 3점

막히면 ▸ 개수가 다르면 '설비별로 따로

막히면 ▸ 계산했는지 확인해 줘'

04

BLOCK 3–4 · 분석

노이즈 vs 추세 · 파일럿 선정

원본 ④ 분석 1 → 보강 실습 B

튐은 노이즈, 기준선을 연속으로 넘는 흐름이 신호다

평균 ± 2σ는 상승 구간까지 '평소'로 계산해 늦다. 정상 구간을 기준선으로 따로 잡는다

단계	규칙
기준선	처음 12점(08:00~11:40) — 정상 운전 구간의 평균과 σ
상한	기준선 평균 + 3σ (한 점 튐에 덜 흔들리게)
추세	상한을 3번 연속 넘으면 추세 — 첫 점이 정비 권고 시점
노이즈	넘었지만 연속 3번이 안 되면 노이즈 (설비A 진동 3.5)



설비B 온도 13:00 · 진동 15:00 — 평균 ± 2σ(15:40)보다 160분 빠르다

이상 후보를 노이즈와 추세로 판정하고 정비 권고 시점을 찾는다

수업 체험에서 본 그 장면이다. 차트에 기준선 상한과 평상 범위 상한을 함께 그린다

① 어디



Code 탭 입력칸 Claude 데스크톱 앱 Code 탭 아래쪽 칸 — 붙여 넣고 Enter

② 입력 · 프롬프트 · Code 탭 입력칸

```
이상 후보가 노이즈인지 추세인지 판정해 줘.
1. 설비 × 센서마다 처음 12점을 기준선으로
   평균 + 3 × 표준편차 = 기준선 상한.
2. 3번 연속 넘으면 '추세'(첫 점 = 추세 시작),
   넘었지만 연속 3번이 안 되면 '노이즈'.
3. trend_judgement.csv 로 저장.
4. 가장 이른 추세 시작과 평균 ± 2σ 첫 후보의
   차이(분).
5. 설비B 진동·온도 차트를 보여 줘.
```

③ 출력 · 화면에 이렇게 나온다

```
설비B 온도  추세 시작 13:00
설비B 진동  추세 시작 15:00
설비A 진동  노이즈 (3.5 한 점)
± 2σ(15:40)보다 160분 빠름
```

④ 피드백 · 확인하고 다음으로

성공 ▸ 추세 2건 · 13:00 · 160분

막히면 ▸ 12:40이 나오면 상한을 3σ로

막히면 ▸ 잡았는지 확인한다

파일럿은 연간 정지손실 ÷ 초기 투자로 고른다

원본 보강 실습 B — 공장장 질문: "예지보전, 우리도 해야 하나? 한다면 어디부터?"

공정	시간당 손실	연 고장 × 정지(h)	연간 정지손실	초기 투자	배수
프레스	1,200	18 × 3.0	64,800	25,000	2.59배
도장	1,500	6 × 4.0	36,000	35,000	1.03배
차체	1,800	24 × 0.8	34,560	60,000	0.58배
조립	2,500	5 × 1.5	18,750	40,000	0.47배

1순위 프레스 — 감축률 25%(추정) 시 연 16,200만원, 회수 1.5년

공정 4곳의 배수로 파일럿을 고르고 순효과를 추정한다

연간 정지손실 전체가 절감액은 아니다. 감축률은 추정으로 밝힌다

① 어디  **Code 탭 입력칸** Claude 데스크톱 앱 Code 탭 아래쪽 칸 — 붙여 넣고 Enter

② 입력 · 프롬프트 · Code 탭 입력칸

```
파일럿 가정값 파일로 파일럿 공정을 골라 줘.
1. 연간 정지손실 = 시간당 × 연간 고장 ×
   평균 정지 시간을 다시 계산해 대조.
2. 연간 정지손실 ÷ 초기 투자(배) 순위.
3. 감축률 25%(추정)로 연간 절감액과
   회수 기간(년).
4. pilot_ranking.csv 저장, 1순위 근거 3줄.
   시간당 손실 기준 순위와도 비교.
```

③ 출력 · 화면에 이렇게 나온다

```
1 프레스 2.59배  절감 16,200  회수 1.5년
2 도장   1.03배
3 차체   0.58배
4 조립   0.47배 (시간당 기준이면 1위)
```

④ 피드백 · 확인하고 다음으로

성공 ▸ 프레스 2.59배 1순위 · '추정' 표기

막히면 ▸ 조립이 1순위면 '연간 정지손실 ÷

막히면 ▸ 투자로 다시 순위를 매겨 줘'

05

BLOCK 5–6 · 배포와 공유

대시보드 공개 · 보고서와 공유

원본 ⑤ 분석 2 → ⑥ 평가 → 동료에게 공유

결과를 index.html 한 장으로 모으고 브라우저로 연다

맨 아래에 '실습용 가상 데이터'와 원본 강의 저장소 링크를 반드시 넣는다

① 어디  **Code 탭 입력칸** Claude 데스크톱 앱 Code 탭 아래쪽 칸 — 붙여 넣고 Enter

② 입력 · 프롬프트 · Code 탭 입력칸

```
결과로 예지보전 대시보드를 만들자.
1. index.html 한 장:
   - 한 줄 요약 (추세 시작·노이즈·파일럿)
   - 설비B 진동·온도 차트, 설비A 진동 차트
   - 판정표 · 파일럿 순위표
   - 도입 사례 표 (출처 링크)
   - '실습용 가상 데이터' 표기 · 원본 링크
   - 휴대폰에서 가로 스크롤 없이
2. 내 브라우저로 열어 줘.
```

③ 출력 · 화면에 이렇게 나온다

```
(브라우저)
설비 예지보전 대시보드
설비B 온도가 13:00부터 추세적으로 상승 …
차트 3개 · 표 3개
```

④ 피드백 · 확인하고 다음으로

성공 ▸ 차트 3개와 표 3개 · 가상 데이터 표기

막히면 ▸ 차트가 안 보이면 회사망 차단 —

막히면 ▸ 개인 네트워크에서 다시 연다

GitHub 연결은 일회용 코드로 하고 코드는 브라우저에만 넣는다

Day3에서 연결했다면 바로 'Logged in'이 나온다

① 어디

브라우저 주소가 정확히 github.com/login/device 인지 먼저 본다

② 입력 · Code 탭 → 브라우저 → 다시 Code 탭

[Code 탭 입력칸]
1. 'gh auth status로 GitHub 연결을 확인하고,
   안 되어 있으면 일회용 코드만 보여 줘'
[브라우저]
2. github.com/login/device 열기
3. 8자리 코드 입력 → Authorize
[Code 탭 입력칸]
4. 승인했어 ← 이렇게 입력

③ 출력 · 화면에 이렇게 나온다

```
! First copy your one-time code:
    ABCD-1234   (예시)
✓ Logged in to github.com account 내아이디
```

④ 피드백 · 확인하고 다음으로

성공 ▸ Logged in ... account 내아이디

막히면 ▸ 코드 만료 → 'GitHub 로그인 다시 시작해 줘'

막히면 ▸ 코드는 남에게 절대 보내지 않는다

저장소를 만들어 올리고 GitHub Pages로 내 주소를 받는다

올라갈 파일 목록을 먼저 보고 승인한다. 켠 뒤 1~3분은 404가 정상이다

① 어디  **Code 탭 입력칸** Claude 데스크톱 앱 Code 탭 아래쪽 칸 — 붙여 넣고 Enter

② 입력 · 프롬프트 · Code 탭 입력칸

```
README.md 와 .gitignore(env_check.txt 제외)를
만들고, 올라갈 파일 목록을 보여 준 뒤
개인정보·비밀번호가 없는지 점검해 줘.

커밋하고 kma-day5-pdm 공개 저장소를 만들어
푸시한 뒤 GitHub Pages(main, /)를 켜 줘.
built가 될 때까지 확인하고
사이트 주소를 크게 보여 줘.
```

③ 출력 · 화면에 이렇게 나온다

```
올라갈 파일: index.html · README.md · data/ ...
status: building ... built ✓
https://내아이디.github.io/kma-day5-pdm/
```

④ 피드백 · 확인하고 다음으로

성공 ▸ 휴대폰에서 대시보드가 열린다

막히면 ▸ 5분 넘게 404 → 'Pages가 404인데 막히면 ▸ 원인을 찾아 줘'

보고서는 4단으로 쓰고 추정은 추정이라고 밝힌다

원본 ⑤ 분석 2의 구성 · 루브릭 이상탐지 30 · 노이즈·추세 25 · 원인 20 · 개선안 25

단	내용	이 실습에서
① 현황 진단	무엇을 어떻게 쟀나	설비 2대 × 25점 · 단위 6칸 변환
② 이상 패턴·원인	노이즈인가 추세인가, 왜	설비A 노이즈 · 설비B 온도 13:00·진동 15:00 추세 · 원인은 추정
③ 정비 권고 시점	언제 무엇을	설비B 당일 점검 권고
④ 개선안	주기·임계값 조정	판정 규칙 변경 · 프레스 파일럿 · 로거 단위 점검

제출 전 — '노이즈를 추세로 잘못 본 곳, 단위를 통일 안 한 곳, 추정 표기가 빠진 곳을 찾아 줘'

보고서를 쓰고 README를 보강한 뒤 공유 메시지를 만든다

공유 메시지는 Claude가 만들고, 보내는 것은 내가 한다

① 어디



Code 탭 입력칸 Claude 데스크톱 앱 Code 탭 아래쪽 칸 — 붙여 넣고 Enter

② 입력 · 프롬프트 · Code 탭 입력칸

```
마무리하자.
1. outputs/pdm_report.md — 4단 보고서,
   추정 표기 · 가상 데이터 표기.
2. 보고서를 거꾸로 검토해 줘.
3. README 보강: 대시보드 주소, 블록별 표,
   핵심 숫자, 원본 강의 출처.
4. 커밋하고 푸시.
5. 동료에게 보낼 공유 메시지 5줄 —
   주소 · 설치 한 줄 · /learn-auto-pdm ·
   가상 데이터. 메시지는 보내지 마.
```

③ 출력 · 화면에 이렇게 나온다

```
pdm_report.md  ① 현황 ② 원인(추정) ③ 정비 ④
개선
[main 4b2e1d] 예지보전 보고서와 README
공유 메시지 5줄 준비 완료
```

④ 피드백 · 확인하고 다음으로

성공 ▸ 보고서 4단 · 푸시 · 공유 메시지

막히면 ▸ 원인에 '추정'이 없으면

막히면 ▸ '추정 표기를 넣어 줘'

공유 메시지 한 통이면 동료가 같은 수업을 듣는다

받은 동료는 설치 한 줄을 Claude Code에 붙여 넣고 /learn-auto-pdm을 입력한다

공유 메시지 (예시)

```
[내 강의 공유] Day5 예지보전을 자율학습 강의로 정리했습니다.
결과 대시보드: https://내아이디.github.io/kma-day5-pdm/
설치 — Claude Code 입력칸에 붙여 넣기:
npx skills add <강의 저장소>/learn-auto-pdm ... 를 실행해 줘
새 세션에서 /learn-auto-pdm → Block 0부터. 데이터는 가상입니다.
```

받은 동료가 확인할 것

- Block 0 env_check 모두 OK
- Block 3 추세 시작 13:00
- Block 4 프레스 2.59배
- 숫자가 다르면 판정 규칙부터 대조

의견을 받으면 고쳐 다시 푸시하고 README 개정 이력에 한 줄 남긴다

막히면 상황을 그대로 Claude Code에게 말한다

아래 문장을 Code 탭 입력칸에 그대로 쓴다

상황	Code 탭 입력칸에 이렇게 쓴다
뭘 해야 할지 모르겠다	지금 뭘 하면 돼?
빨간 오류 글자가 나왔다	방금 오류 고쳐 줘
숫자가 매뉴얼과 다르다	결과가 가이드와 다른데 판정 규칙부터 대조해서 원인을 찾아 줘
GitHub 연결 코드가 만료됐다	GitHub 로그인을 처음부터 다시 시작해 줘
사이트가 404 (3분 이상)	Pages가 404인데 원인을 찾아 줘
사이트가 예전 화면 그대로	커밋하고 푸시까지 했는지 확인해 줘
도저히 안 된다	정답 보기

제출물은 주소·보고서·공유 메시지, 과제는 세 가지다

과제는 Block 6 퀴즈가 끝나면 Claude가 화면에 보여 준다

1

동료 한 명에게 공유

공유 메시지를 실제로 보내고, 동료가 Block 0을 통과했는지 확인 · 받은 의견 한 가지를 README 개정 이력에

2

보강 실습 A를 내 설비로

고장 모드 × 센서 × 주기 × 경보 3등급 감시 설계표 — 실제 값은 '추정', 비공개 저장소에만

3

내 강의 하나 더

/course-forge 로 Day4 업무 자동화를 내 강의로 만들어 배포하고 공유한다

공개 저장소에는 공개해도 되는 자료만 — 회사 자료는 회사가 승인한 비공개 저장소에

B

부록 · VS Code

VS Code로 따라 하기

데스크톱 앱 대신 VS Code 터미널에서 Claude Code를 쓰는 경우

VS Code로 할 때는 다섯 군데만 다르다

프롬프트·블록 순서·확인할 숫자는 본문과 같다. 설치는 맥 curl · 윈도우 PowerShell irm 한 줄

상황	데스크톱 앱 (본문)	VS Code
입력하는 곳	Code 탭 입력칸	아래 터미널의 > 옆
작업 폴더 열기	새 세션 → Local → 폴더 선택	/exit → 파일 → 폴더 열기 → 새 터미널 → claude
계획 모드	모드 선택기 → Plan	Shift + Tab → ⏸ plan mode on
GitHub 로그인 직접	Ctrl+` 앱 안 터미널	입력칸에 ! gh auth login --web
멈추기	정지 버튼	Esc 키